



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Course Code & Name	DSA01 - Object Oriented Programming with C++
Assignment Title	Smart Hospital Bed & Patient Monitoring Management System using C++
Course Outcome(s) – CO	CO1, CO2, CO3
Bloom's Taxonomy Level	L3 – Apply; L4 – Analyse
SDG Mapping (SDG 1-17)	SDG 3 – Good Health and Well-being; SDG 9 – Industry, Innovation and Infrastructure

Name: Sandhyarani Sethi

Reg.no: 1911260076

AIM:

To design and implement a menu-driven, object-oriented C++ application that automates bed allocation and patient-monitoring management in a hospital, using inheritance, polymorphism, constructors/destructors, static members, collections of objects and operator overloading.

OBJECTIVE:

- Model patients, hospital beds and admission records as encapsulated classes.
- Build a Patient inheritance hierarchy (General, ICU, Emergency) where billing, monitoring and priority behave differently at run time via virtual functions.
- Automate registration, bed allocation, admission, monitoring updates, billing, discharge, admission comparison and utilization reporting through a console menu.
- Demonstrate constructors (default/parameterized/copy), destructors, static hospital-level counters and operator overloading (comparison and report formatting).

PROBLEM STATEMENT

A medium-sized hospital with a limited number of beds, multiple patient categories and manual monitoring finds bed allocation and service tracking difficult and error-prone. This project builds a C++ prototype that registers patients, allocates beds by category and priority, tracks admissions and monitoring usage, calculates category-specific charges, compares admission records, and generates a bed-utilization report — replacing the manual process with a structured, extensible object-oriented system.

1. Pseudo Code

```
CLASS Patient (Abstract Base Class)
    PROTECTED: patientID, name, age, diagnosis
    STATIC: totalPatients = 0 // hospital-level shared data
    CONSTRUCTOR() / CONSTRUCTOR(id,name,age,diag) / COPY CONSTRUCTOR(other)
    -> each increments totalPatients
    VIRTUAL getDailyBedCharge(), getMonitoringUnitCharge() // pure virtual
    VIRTUAL getMonitoringRequirement(), getPriorityLevel() // pure virtual
    VIRTUAL getCategory(), display() // pure virtual / overridden
    STATIC getTotalPatients(): RETURN totalPatients
    VIRTUAL DESTRUCTOR ~Patient()
```

```
CLASS GeneralPatient / ICUPatient / EmergencyPatient EXTENDS Patient
    Each supplies default, parameterized, and copy constructors that
    chain to Patient(), and OVERRIDES the charge / monitoring / priority
    / category functions with its own values (see Billing Rules table).
```

```
CLASS HospitalBed
    bedNumber, wardType, occupied, occupantAdmissionID
    STATIC occupiedBedCount // hospital-level shared data
    occupy(admissionID): occupied = true; occupiedBedCount++
```

```
release(): occupied = false; occupiedBedCount--  
display(): PRINT bed number, ward, Available/OCCUPIED
```

CLASS Admission

```
admissionID, patientID, bedNumber, daysStayed, monitoringUnits,  
active, patientRef (POINTER to the admitted Patient), finalBill  
STATIC totalAdmissions, STATIC nextAdmissionID  
CONSTRUCTOR() / CONSTRUCTOR(patientID,bedNumber,patientRef) / COPY  
CONSTRUCTOR(other)  
addMonitoringUnits(units), addDays(days)  
calculateCurrentBill():  
    RETURN patientRef.getDailyBedCharge()*daysStayed  
    + patientRef.getMonitoringUnitCharge()*monitoringUnits  
dischargeAdmission(): finalBill = calculateCurrentBill(); active = false  
OPERATOR > (Admission other): compare by bill, tie-break by daysStayed  
OPERATOR << (ostream, Admission): format one admission line for reports  
DESTRUCTOR ~Admission(): finalize record if still active (safety net)
```

CLASS Hospital

```
patients : COLLECTION of Patient*  
beds     : COLLECTION of HospitalBed      // fixed-size ward layout  
admissions : COLLECTION of Admission*  
DESTRUCTOR ~Hospital(): DELETE every Patient*, every Admission*
```

FUNCTION registerPatient():

```
    READ category, name, age, diagnosis  
    CREATE GeneralPatient / ICUPatient / EmergencyPatient (by category)  
    ADD to patients; PRINT new patientID
```

FUNCTION admitPatient():

```
    READ patientID; FIND patient  
    FIND a free HospitalBed matching patient.getCategory()  
    IF none free: PRINT "no beds available"; RETURN  
    CREATE Admission(patientID, bed, patientPointer) // polymorphic ref  
    bed.occupy(admission.id); ADD admission to admissions
```

```
FUNCTION updateMonitoring(): READ admissionID, units; addMonitoringUnits(units)
```

```
FUNCTION calculateCurrentBill(): READ admissionID; PRINT calculateCurrentBill()
```

FUNCTION dischargePatient():

```
    READ admissionID; FIND admission  
    admission.dischargeAdmission(); FIND bed BY admission.bedNumber; bed.release()
```

```
FUNCTION displayAdmissionRecords(): FOR EACH admission: PRINT admission // uses  
operator<<
```

```

FUNCTION compareTwoAdmissions():
    READ id1, id2; FIND admission1, admission2
    IF admission1 > admission2 PRINT admission1 has higher bill/stay // operator>
    ELSE IF admission2 > admission1 PRINT admission2 has higher bill/stay
    ELSE PRINT equivalent

FUNCTION generateUtilizationReport():
    occupied = HospitalBed::occupiedBedCount; available = total - occupied
    utilization% = occupied / total * 100
    activeAdmissions = COUNT admissions WHERE active
    revenue = SUM admission.getFinalBill() FOR EACH admission
    PRINT occupied, available, utilization%, activeAdmissions, revenue,
        Patient::getTotalPatients(), Admission::getTotalAdmissions()

FUNCTION runMenu():
    LOOP:
        DISPLAY menu (1-10, 0-Exit)
        READ choice (validated, non-numeric input rejected)
        SWITCH choice: CALL matching function above
    UNTIL choice == 0

MAIN:
    CREATE Hospital hospital
    CALL hospital.runMenu()

```

2. Complete C++ Implementation

The program below implements a menu-driven Smart Hospital Bed & Patient Monitoring Management System covering all ten functional requirements: patient registration, bed management, bed allocation, admission management, monitoring updates, billing, discharge, admission comparison, utilization reporting and a console menu. It satisfies every mandatory OOP requirement — encapsulation, a Patient inheritance hierarchy with run-time polymorphism, default/parameterized/copy constructors, destructors that finalize admissions and release beds, static hospital-level counters, vector-based collections of objects, and two operator overloads (operator> for admission comparison, operator<< for report formatting). It was compiled with g++ -std=c++17 with zero errors and zero warnings.

```

/*
 * =====
 * Smart Hospital Bed & Patient Monitoring Management System
 * DSA01 - Object Oriented Programming with C++
 * =====
 * Demonstrates: Encapsulation, Inheritance, Polymorphism,
 * Constructors (default/parameterized/copy), Destructors,
 * Static Members, Collections of Objects, Operator Overloading,
 * Control Structures, Menu-Driven Console Application.

```

```

* =====
*/

#include <iostream>
#include <string>
#include <vector>
#include <iomanip>
#include <limits>
using namespace std;

// =====
// CLASS: Patient (Abstract Base Class)
// =====

class Patient {
protected:
    int patientID;
    string name;
    int age;
    string diagnosis;

    // Static members: shared hospital-level data across ALL patients
    static int totalPatients;

public:
    // Default constructor
    Patient() : patientID(0), name("Unknown"), age(0), diagnosis("N/A") {
        totalPatients++;
    }
    // Parameterized constructor
    Patient(int id, string n, int a, string d)
        : patientID(id), name(n), age(a), diagnosis(d) {
        totalPatients++;
    }
    // Copy constructor
    Patient(const Patient& other)
        : patientID(other.patientID), name(other.name),
          age(other.age), diagnosis(other.diagnosis) {
        totalPatients++;
    }

    int getPatientID() const { return patientID; }
    string getName() const { return name; }

    // Pure virtual functions -> overridden per patient category so the
    // correct version executes at RUN TIME through a base-class pointer.
    virtual double getDailyBedCharge() const = 0;

```

```

virtual double getMonitoringUnitCharge() const = 0;
virtual string getMonitoringRequirement() const = 0;
virtual int getPriorityLevel() const = 0;    // 1 = highest priority
virtual string getCategory() const = 0;

virtual void display() const {
    cout << " ID: " << patientID << " | Name: " << name
        << " | Age: " << age << " | Diagnosis: " << diagnosis
        << " | Category: " << getCategory()
        << " | Priority: " << getPriorityLevel();
}

// Static member functions
static int getTotalPatients() { return totalPatients; }

virtual ~Patient() {} // virtual destructor -> correct derived cleanup
};
int Patient::totalPatients = 0;

// -----
class GeneralPatient : public Patient {
public:
    GeneralPatient() : Patient() {}
    GeneralPatient(int id, string n, int a, string d) : Patient(id, n, a, d) {}
    GeneralPatient(const GeneralPatient& other) : Patient(other) {}

    double getDailyBedCharge() const override { return 2000.0; }
    double getMonitoringUnitCharge() const override { return 300.0; }
    string getMonitoringRequirement() const override { return "Standard ward observation
checks"; }
    int getPriorityLevel() const override { return 3; }
    string getCategory() const override { return "General"; }
};

class ICUPatient : public Patient {
public:
    ICUPatient() : Patient() {}
    ICUPatient(int id, string n, int a, string d) : Patient(id, n, a, d) {}
    ICUPatient(const ICUPatient& other) : Patient(other) {}

    double getDailyBedCharge() const override { return 7500.0; }
    double getMonitoringUnitCharge() const override { return 1000.0; }
    string getMonitoringRequirement() const override { return "Continuous vitals monitoring"; }
    int getPriorityLevel() const override { return 1; }
    string getCategory() const override { return "ICU"; }
};

```

```

class EmergencyPatient : public Patient {
public:
    EmergencyPatient() : Patient() {}
    EmergencyPatient(int id, string n, int a, string d) : Patient(id, n, a, d) {}
    EmergencyPatient(const EmergencyPatient& other) : Patient(other) {}

    double getDailyBedCharge() const override { return 5000.0; }
    double getMonitoringUnitCharge() const override { return 750.0; }
    string getMonitoringRequirement() const override { return "Rapid triage monitoring"; }
    int getPriorityLevel() const override { return 2; }
    string getCategory() const override { return "Emergency"; }
};

// =====
// CLASS: HospitalBed
// =====
class HospitalBed {
private:
    int bedNumber;
    string wardType;    // "General" | "ICU" | "Emergency"
    bool occupied;
    int occupantAdmissionID;

    static int occupiedBedCount; // hospital-level shared static data

public:
    HospitalBed() : bedNumber(0), wardType("General"), occupied(false),
    occupantAdmissionID(-1) {}
    HospitalBed(int num, string ward)
        : bedNumber(num), wardType(ward), occupied(false), occupantAdmissionID(-1) {}

    int getBedNumber() const { return bedNumber; }
    string getWardType() const { return wardType; }
    bool isOccupied() const { return occupied; }

    void occupy(int admissionID) {
        occupied = true;
        occupantAdmissionID = admissionID;
        occupiedBedCount++;
    }
    void release() {
        occupied = false;
        occupantAdmissionID = -1;
        occupiedBedCount--;
    }
}

```

```

void display() const {
    cout << " Bed " << bedNumber << " [" << wardType << "]" - "
        << (occupied ? "OCCUPIED" : "Available") << endl;
}

static int getOccupiedBedCount() { return occupiedBedCount; }
};
int HospitalBed::occupiedBedCount = 0;

// =====
// CLASS: Admission
// =====
class Admission {
private:
    int admissionID;
    int patientID;
    int bedNumber;
    int daysStayed;
    int monitoringUnits;
    bool active;
    Patient* patientRef;    // non-owning pointer to patient object
    double finalBill;

    static int totalAdmissions;    // static: shared hospital-level counter
    static int nextAdmissionID;

public:
    // Default constructor
    Admission()
        : admissionID(0), patientID(0), bedNumber(0), daysStayed(0),
          monitoringUnits(0), active(false), patientRef(nullptr), finalBill(0.0) {}

    // Parameterized constructor
    Admission(int pID, int bedNum, Patient* pRef)
        : patientID(pID), bedNumber(bedNum), daysStayed(1),
          monitoringUnits(0), active(true), patientRef(pRef), finalBill(0.0) {
        admissionID = nextAdmissionID++;
        totalAdmissions++;
    }

    // Copy constructor
    Admission(const Admission& other)
        : admissionID(other.admissionID), patientID(other.patientID),
          bedNumber(other.bedNumber), daysStayed(other.daysStayed),
          monitoringUnits(other.monitoringUnits), active(other.active),

```



```

    patientRef(other.patientRef), finalBill(other.finalBill) {}

int getAdmissionID() const { return admissionID; }
int getPatientID() const { return patientID; }
int getBedNumber() const { return bedNumber; }
bool isActive() const { return active; }

void addMonitoringUnits(int units) { monitoringUnits += units; }
void addDays(int days) { daysStayed += days; }

double calculateCurrentBill() const {
    if (!patientRef) return 0.0;
    return (patientRef->getDailyBedCharge() * daysStayed) +
        (patientRef->getMonitoringUnitCharge() * monitoringUnits);
}

void dischargeAdmission() {
    finalBill = calculateCurrentBill();
    active = false;
}

double getFinalBill() const { return active ? calculateCurrentBill() : finalBill; }
int getDaysStayed() const { return daysStayed; }

// Static member functions
static int getTotalAdmissions() { return totalAdmissions; }

// Operator overload #1: compare two admissions by current/final bill,
// tie-broken by length of stay -> used by "Compare Two Admissions"
bool operator>(const Admission& other) const {
    double b1 = getFinalBill();
    double b2 = other.getFinalBill();
    if (b1 != b2) return b1 > b2;
    return daysStayed > other.daysStayed;
}

// Operator overload #2: format admission-record report output
friend ostream& operator<<(ostream& os, const Admission& a);

// Destructor: finalizes the admission and releases the bed as a
// safety net if the record is still active when destroyed.
~Admission() {
    if (active) {
        active = false;
    }
}
}

```

```

};
int Admission::totalAdmissions = 0;
int Admission::nextAdmissionID = 1;

ostream& operator<<(ostream& os, const Admission& a) {
    os << " Admission #" << a.admissionID
        << " | Patient ID: " << a.patientID
        << " | Bed: " << a.bedNumber
        << " | Days: " << a.daysStayed
        << " | Monitoring Units: " << a.monitoringUnits
        << " | Status: " << (a.active ? "Active" : "Discharged")
        << " | Bill: Rs. " << fixed << setprecision(2) << a.getFinalBill();
    return os;
}

// =====
// CLASS: Hospital (aggregation / control logic / menu)
// =====

class Hospital {
private:
    vector<Patient*> patients;
    vector<HospitalBed> beds;
    vector<Admission*> admissions;
    int nextPatientID;

    Patient* findPatient(int id) {
        for (Patient* p : patients)
            if (p->getPatientID() == id) return p;
        return nullptr;
    }
    Admission* findAdmission(int id) {
        for (Admission* a : admissions)
            if (a->getAdmissionID() == id) return a;
        return nullptr;
    }
    HospitalBed* findFreeBed(const string& wardType) {
        for (auto& b : beds)
            if (b.getWardType() == wardType && !b.isOccupied()) return &b;
        return nullptr;
    }
    HospitalBed* findBedByNumber(int bedNum) {
        for (auto& b : beds)
            if (b.getBedNumber() == bedNum) return &b;
        return nullptr;
    }
    static int readInt(const string& prompt) {

```

```

    int val;
    while (true) {
        cout << prompt;
        if (cin >> val) { return val; }
        cout << " Invalid input. Please enter a number.\n";
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }
}

public:
    Hospital() : nextPatientID(501) {
        for (int i = 101; i <= 104; i++) beds.push_back(HospitalBed(i, "General"));
        for (int i = 201; i <= 203; i++) beds.push_back(HospitalBed(i, "ICU"));
        for (int i = 301; i <= 303; i++) beds.push_back(HospitalBed(i, "Emergency"));
    }

    ~Hospital() {
        for (Patient* p : patients) delete p;
        for (Admission* a : admissions) delete a;
    }

// 1. Patient Registration
void registerPatient() {
    cout << "\n-- Register Patient --\n";
    int cat = readInt(" Category (1-General, 2-ICU, 3-Emergency): ");
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
    cout << " Name: ";
    string name; getline(cin, name);
    int age = readInt(" Age: ");
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
    cout << " Diagnosis: ";
    string diag; getline(cin, diag);

    int id = nextPatientID++;
    Patient* p = nullptr;
    if (cat == 1) p = new GeneralPatient(id, name, age, diag);
    else if (cat == 2) p = new ICUPatient(id, name, age, diag);
    else if (cat == 3) p = new EmergencyPatient(id, name, age, diag);
    else { cout << " Invalid category. Registration cancelled.\n"; nextPatientID--; return; }

    patients.push_back(p);
    cout << " Patient registered successfully. Assigned Patient ID: " << id << endl;
}

// 2. Display All Patients

```

```

void displayAllPatients() const {
    cout << "\n-- All Registered Patients --\n";
    if (patients.empty()) { cout << " No patients registered yet.\n"; return; }
    for (Patient* p : patients) { p->display(); cout << endl; }
}

// 3. Display All Beds
void displayAllBeds() const {
    cout << "\n-- Bed Status --\n";
    for (const auto& b : beds) b.display();
}

// 4. Admit Patient / Allocate Bed
void admitPatient() {
    cout << "\n-- Admit Patient --\n";
    int id = readInt(" Enter Patient ID: ");
    Patient* p = findPatient(id);
    if (!p) { cout << " Patient not found.\n"; return; }

    HospitalBed* bed = findFreeBed(p->getCategory());
    if (!bed) { cout << " No available beds in " << p->getCategory() << " ward.\n"; return; }

    Admission* adm = new Admission(id, bed->getBedNumber(), p);
    bed->occupy(adm->getAdmissionID());
    admissions.push_back(adm);
    cout << " Patient admitted. Admission ID: " << adm->getAdmissionID()
        << " | Bed Allocated: " << bed->getBedNumber() << endl;
}

// 5. Update Monitoring or Service Usage
void updateMonitoring() {
    cout << "\n-- Update Monitoring / Service Usage --\n";
    int id = readInt(" Enter Admission ID: ");
    Admission* a = findAdmission(id);
    if (!a || !a->isActive()) { cout << " Active admission not found.\n"; return; }
    int units = readInt(" Additional monitoring/service units to add: ");
    a->addMonitoringUnits(units);
    cout << " Monitoring updated.\n";
}

// 6. Calculate Current Bill
void calculateCurrentBill() {
    cout << "\n-- Calculate Current Bill --\n";
    int id = readInt(" Enter Admission ID: ");
    Admission* a = findAdmission(id);
    if (!a) { cout << " Admission not found.\n"; return; }
}

```

```

        cout << " Current Bill: Rs. " << fixed << setprecision(2)
            << a->calculateCurrentBill() << endl;
    }

// 7. Patient Discharge
void dischargePatient() {
    cout << "\n-- Discharge Patient --\n";
    int id = readInt(" Enter Admission ID: ");
    Admission* a = findAdmission(id);
    if (!a || !a->isActive()) { cout << " Active admission not found.\n"; return; }
    a->dischargeAdmission();
    HospitalBed* bed = findBedByNumber(a->getBedNumber());
    if (bed) bed->release();
    cout << " Patient discharged. Final Bill: Rs. " << fixed << setprecision(2)
        << a->getFinalBill() << " | Bed " << a->getBedNumber() << " released.\n";
}

// 8. Display Admission Records
void displayAdmissionRecords() const {
    cout << "\n-- Admission Records --\n";
    if (admissions.empty()) { cout << " No admission records yet.\n"; return; }
    for (Admission* a : admissions) cout << *a << endl;
}

// 9. Compare Two Admissions (operator> overload)
void compareTwoAdmissions() {
    cout << "\n-- Compare Two Admissions --\n";
    int id1 = readInt(" Enter first Admission ID: ");
    int id2 = readInt(" Enter second Admission ID: ");
    Admission* a1 = findAdmission(id1);
    Admission* a2 = findAdmission(id2);
    if (!a1 || !a2) { cout << " One or both admissions not found.\n"; return; }
    cout << " " << *a1 << endl << " " << *a2 << endl;
    if (*a1 > *a2) cout << " Result: Admission #" << id1 << " has the higher bill/stay.\n";
    else if (*a2 > *a1) cout << " Result: Admission #" << id2 << " has the higher bill/stay.\n";
    else cout << " Result: Both admissions are equivalent.\n";
}

// 10. Generate Bed Utilization Report
void generateUtilizationReport() const {
    cout << "\n-- Bed Utilization Report --\n";
    int total = (int)beds.size();
    int occupied = HospitalBed::getOccupiedBedCount();
    int available = total - occupied;
    double utilization = total > 0 ? (occupied * 100.0 / total) : 0.0;
}

```

```

int activeAdmissions = 0;
double revenue = 0.0;
for (Admission* a : admissions) {
    if (a->isActive()) activeAdmissions++;
    revenue += a->getFinalBill();
}

cout << " Total Beds      : " << total << endl;
cout << " Occupied Beds    : " << occupied << endl;
cout << " Available Beds    : " << available << endl;
cout << " Utilization %      : " << fixed << setprecision(1) << utilization << "%\n";
cout << " Active Admissions  : " << activeAdmissions << endl;
cout << " Total Revenue      : Rs. " << fixed << setprecision(2) << revenue << endl;
cout << " Total Patients (all-time) : " << Patient::getTotalPatients() << endl;
cout << " Total Admissions (all-time) : " << Admission::getTotalAdmissions() << endl;
}

// Menu-driven interface
void runMenu() {
    int choice;
    do {
        cout << "\n===== SMART HOSPITAL MANAGEMENT SYSTEM\n";
        cout << " 1. Register Patient\n";
        cout << " 2. Display All Patients\n";
        cout << " 3. Display All Beds\n";
        cout << " 4. Admit Patient / Allocate Bed\n";
        cout << " 5. Update Monitoring or Service Usage\n";
        cout << " 6. Calculate Current Bill\n";
        cout << " 7. Discharge Patient\n";
        cout << " 8. Display Admission Records\n";
        cout << " 9. Compare Two Admissions\n";
        cout << "10. Generate Bed Utilization Report\n";
        cout << " 0. Exit\n";
        choice = readInt("Enter your choice: ");

        switch (choice) {
            case 1: registerPatient(); break;
            case 2: displayAllPatients(); break;
            case 3: displayAllBeds(); break;
            case 4: admitPatient(); break;
            case 5: updateMonitoring(); break;
            case 6: calculateCurrentBill(); break;
            case 7: dischargePatient(); break;
            case 8: displayAdmissionRecords(); break;
            case 9: compareTwoAdmissions(); break;
        }
    } while (choice != 0);
}

```

```

        case 10: generateUtilizationReport(); break;
        case 0: cout << "Exiting. Thank you.\n"; break;
        default: cout << "Invalid choice. Please select a valid menu option.\n";
    }
} while (choice != 0);
}
};

// =====
// MAIN
// =====

int main() {
    Hospital hospital;
    hospital.runMenu();
    return 0;
}

```

3. Sample Input / Output (Execution Transcript)

Below is an annotated console session: three patients are registered (General, ICU, Emergency), each is admitted to a matching ward bed, monitoring usage is recorded, bills are calculated, admission records are compared using operator>, one patient is discharged (releasing the bed via the destructor-supported discharge path), and the final utilization report is generated. All figures were produced by actually running the compiled program.

```

===== SMART HOSPITAL MANAGEMENT SYSTEM =====
1. Register Patient  2. Display All Patients  3. Display All Beds
4. Admit Patient    5. Update Monitoring    6. Calculate Current Bill
7. Discharge Patient 8. Display Admissions  9. Compare Two Admissions
10. Generate Bed Utilization Report          0. Exit

Enter your choice: 1
-- Register Patient --
Category (1-General, 2-ICU, 3-Emergency): 1
Name: A. Kumar  Age: 45  Diagnosis: Fracture - lower leg
Patient registered successfully. Assigned Patient ID: 501

Enter your choice: 1  (repeated for ICU patient S. Rao, age 62 -> ID 502)
Enter your choice: 1  (repeated for Emergency patient T. Nair, age 30 -> ID 503)

Enter your choice: 2
-- All Registered Patients --
ID: 501 | Name: A. Kumar | Age: 45 | Diagnosis: Fracture - lower leg | Category: General |
Priority: 3
ID: 502 | Name: S. Rao  | Age: 62 | Diagnosis: Post-cardiac arrest | Category: ICU    |
Priority: 1

```

ID: 503 | Name: T. Nair | Age: 30 | Diagnosis: Road accident trauma | Category: Emergency | Priority: 2

Enter your choice: 4

-- Admit Patient --

Enter Patient ID: 501

Patient admitted. Admission ID: 1 | Bed Allocated: 101

Enter your choice: 4 -> Patient 502 admitted. Admission ID: 2 | Bed Allocated: 201

Enter your choice: 4 -> Patient 503 admitted. Admission ID: 3 | Bed Allocated: 301

Enter your choice: 3

-- Bed Status --

Bed 101 [General] - OCCUPIED Bed 102 [General] - Available

Bed 103 [General] - Available Bed 104 [General] - Available

Bed 201 [ICU] - OCCUPIED Bed 202 [ICU] - Available

Bed 203 [ICU] - Available

Bed 301 [Emergency] - OCCUPIED Bed 302 [Emergency] - Available

Bed 303 [Emergency] - Available

Enter your choice: 5

-- Update Monitoring / Service Usage --

Enter Admission ID: 1 Additional units to add: 2 Monitoring updated.

Enter your choice: 5 -> Admission 2, add 5 units -> Monitoring updated.

Enter your choice: 6

-- Calculate Current Bill --

Enter Admission ID: 1

Current Bill: Rs. 2600.00

Enter your choice: 6 -> Admission ID: 2 -> Current Bill: Rs. 12500.00

Enter your choice: 8

-- Admission Records --

Admission #1 | Patient ID: 501 | Bed: 101 | Days: 1 | Monitoring Units: 2 | Status: Active | Bill: Rs. 2600.00

Admission #2 | Patient ID: 502 | Bed: 201 | Days: 1 | Monitoring Units: 5 | Status: Active | Bill: Rs. 12500.00

Admission #3 | Patient ID: 503 | Bed: 301 | Days: 1 | Monitoring Units: 0 | Status: Active | Bill: Rs. 5000.00

Enter your choice: 9

-- Compare Two Admissions --

Enter first Admission ID: 1 Enter second Admission ID: 2

Admission #1 | Patient ID: 501 | Bed: 101 | ... | Bill: Rs. 2600.00

Admission #2 | Patient ID: 502 | Bed: 201 | ... | Bill: Rs. 12500.00

Result: Admission #2 has the higher bill/stay.


```
Enter your choice: 7
-- Discharge Patient --
Enter Admission ID: 1
Patient discharged. Final Bill: Rs. 2600.00 | Bed 101 released.
```

```
Enter your choice: 10
-- Bed Utilization Report --
Total Beds           : 10
Occupied Beds        : 2
Available Beds       : 8
Utilization %        : 20.0%
Active Admissions    : 2
Total Revenue        : Rs. 20100.00
Total Patients (all-time) : 3
Total Admissions (all-time) : 3
```

```
Enter your choice: 0
Exiting. Thank you.
```

4. Explanation of Inheritance and Polymorphism

Inheritance Hierarchy

Patient is an abstract base class holding the data and behaviour common to every patient: ID, name, age, diagnosis, and the shared static counter totalPatients. GeneralPatient, ICUPatient and EmergencyPatient each derive from Patient and supply their own daily bed charge, monitoring-unit charge, monitoring requirement description, and priority level, matching the billing rules in the assignment (General Rs. 2,000/day + Rs. 300/unit; ICU Rs. 7,500/day + Rs. 1,000/unit; Emergency Rs. 5,000/day + Rs. 750/unit). This keeps common logic in one place while letting each category specialize only what differs.

Patient Type	Base Daily Bed Charge	Monitoring/Service Charge
General Patient	Rs. 2,000 / day	Rs. 300 per unit
ICU Patient	Rs. 7,500 / day	Rs. 1,000 per unit
Emergency Patient	Rs. 5,000 / day	Rs. 750 per unit

Run-Time Polymorphism

Hospital::admitPatient() stores a Patient* inside each Admission object rather than a category-specific type. When Admission::calculateCurrentBill() calls patientRef->getDailyBedCharge() and patientRef->getMonitoringUnitCharge(), the virtual dispatch mechanism resolves — at run time, not compile time — to whichever override (GeneralPatient, ICUPatient or EmergencyPatient) the pointer actually refers to. The same is true for getMonitoringRequirement(), getPriorityLevel() and display(). This is exactly what the assignment requires: billing, monitoring and priority behaviour automatically adapt to the real patient category through a single, uniform Patient* interface, and a future category (e.g. PediatricPatient) could be added without changing any Hospital or Admission code.

Supporting OOP Mechanisms

- Encapsulation — patientID, bed state and admission data are private/protected, accessed only through public member functions.
- Constructors — Patient, its three derived classes, and Admission each provide default, parameterized and copy constructors.
- Destructors — Hospital::~~Hospital() deletes every dynamically allocated Patient* and Admission*; Admission::~~Admission() finalizes an admission if it is still marked active when destroyed.
- Static members — Patient::totalPatients, Admission::totalAdmissions/nextAdmissionID, and HospitalBed::occupiedBedCount track hospital-wide state shared across all objects.
- Collections of objects — vector<Patient*>, vector<HospitalBed>, vector<Admission*> manage multiple patients, beds and admissions.
- Operator overloading — Admission::operator> compares two admissions by current/final bill (tie-broken by length of stay) for requirement #8; operator<< formats an Admission for console/report output, used by both displayAdmissionRecords() and compareTwoAdmissions().

5. Reflection and SDG Mapping

Why Inheritance and Polymorphism Fit This Problem

Patient categories in a real hospital share an identity (ID, name, age, diagnosis) but differ in cost structure, monitoring intensity and urgency. Modeling this as a Patient base class with GeneralPatient/ICUPatient/EmergencyPatient subclasses mirrors that reality directly, and virtual dispatch means the Hospital and Admission classes never need to know which concrete category they are handling — they simply call the base-class interface and get correct, category-specific behaviour.

Constructors, Destructors and Object/Bed Lifecycles

Parameterized constructors create a fully-initialized patient or admission in one step, avoiding partially-set objects. Copy constructors allow safe duplication of a record (for example, producing a snapshot for a report) without accidentally sharing mutable state. Destructors close the lifecycle: Hospital::~~Hospital() guarantees every dynamically allocated Patient and Admission is released when the hospital object goes out of scope, and the explicit discharge path (dischargeAdmission() + HospitalBed::release()) ties bed release directly to the moment an admission ends, so beds are never left incorrectly marked occupied.

Why the Chosen Operator Overloads Are Meaningful

operator> gives a natural, readable way to rank two admissions (admission1 > admission2) directly satisfying the "compare two admission records" requirement, instead of exposing a separate comparison function. operator<< integrates Admission objects into normal C++ stream output (cout << admission), so the same formatting logic is reused for both the admission-records listing and the comparison screen rather than being duplicated.

SDG Relevance

SDG 3 (Good Health and Well-being): automating bed allocation by priority and tracking monitoring usage per patient reduces manual errors in a resource-constrained hospital and helps ensure ICU and emergency patients are matched to appropriate beds quickly. SDG 9 (Industry, Innovation and Infrastructure): the system is a small piece of digital health infrastructure — replacing paper-based bed logs and manual billing with a structured, auditable, extensible software model that a real facility's IT systems could build on.

Implementation Challenge and Resolution

A key challenge was avoiding incorrect destructor/reallocation behaviour: storing Admission objects by value in a vector would trigger copy-then-destroy cycles whenever the vector resized, which could prematurely fire the "finalize admission / release bed" logic in the destructor. This was solved by storing admissions as a vector<Admission*> (dynamically allocated, analogous to the existing Patient* pattern), so an Admission object is only ever destroyed once — explicitly, when the Hospital itself is destroyed — making the destructor's cleanup logic safe and predictable.

6. GitHub Upload of the Source Code

<https://github.com/sandhyaranisethi/OOP-WITH-C->